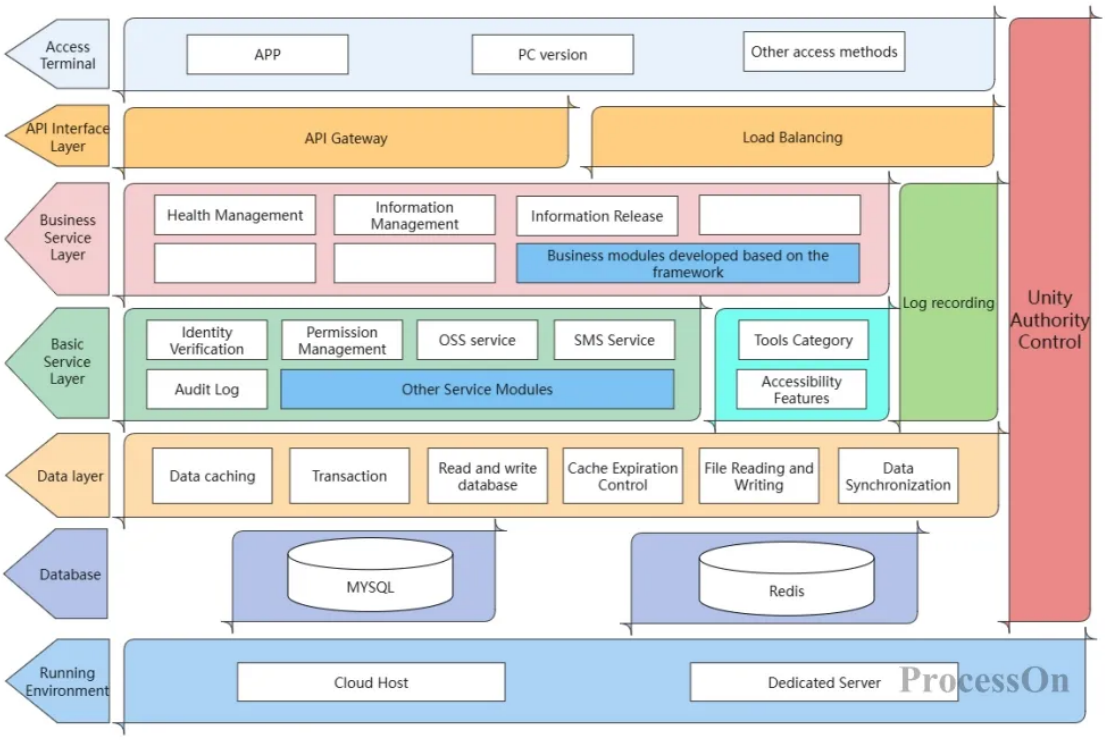
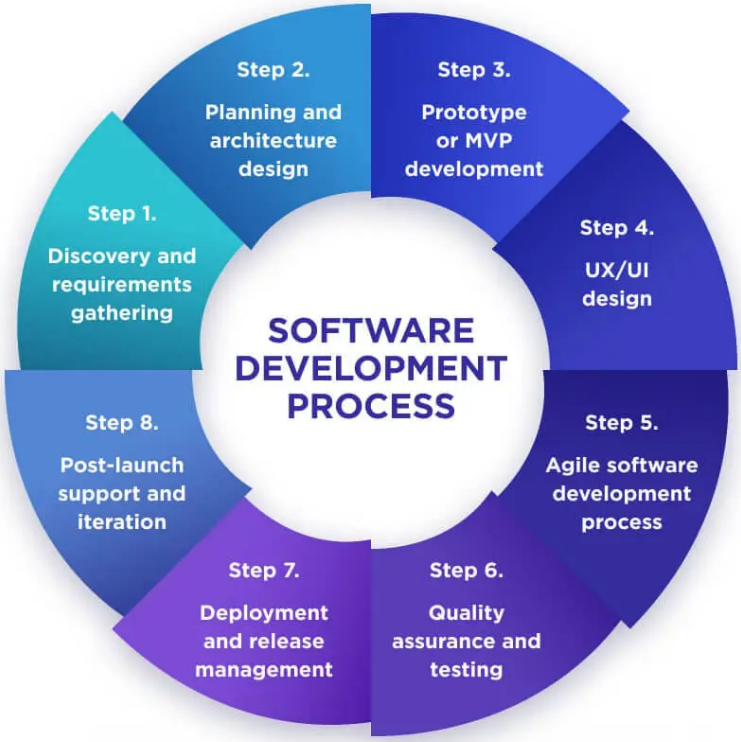
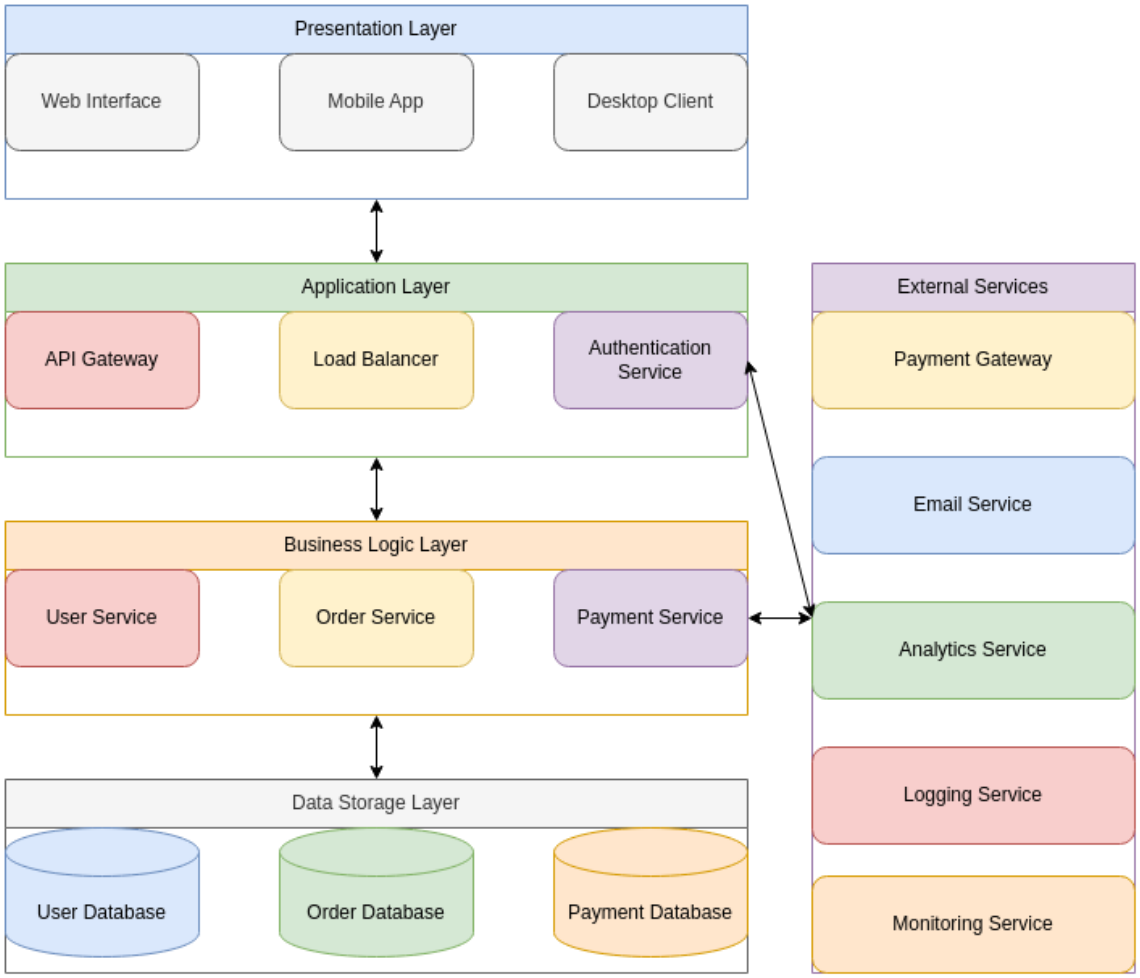
Sesión 23
UML: Introducción y elementos de los diagramas de clases

¿Por qué modelar antes de programar?





Luz María Álvarez Moreno



En desarrollo de software es habitual querer **empezar a programar inmediatamente**. Sin embargo, en proyectos reales primero se realiza **modelado del sistema**.

Modelado de software: proceso de representar un sistema mediante diagramas o modelos para comprender su estructura y funcionamiento antes de implementarlo.

El modelado permite:

- entender el sistema antes de programar
- comunicar ideas dentro del equipo
- detectar errores de diseño
- organizar el código antes de escribirlo

El material explica que visualizar la arquitectura antes de programar evita errores como duplicar lógica o crear dependencias innecesarias .

UML (Unified Modeling Language)

UML (Lenguaje Unificado de Modelado) es un lenguaje estándar utilizado para **representar gráficamente sistemas de software**.
Fue creado en los años 90 para unificar diferentes métodos de modelado.

Objetivos de UML

- representar sistemas complejos
- facilitar el diseño orientado a objetos
- servir como lenguaje común entre desarrolladores
- documentar arquitectura

Tipos de diagramas UML

Tipo de diagrama	Qué representa
Diagrama de clases	estructura del sistema
Diagrama de objetos	instancias concretas
Diagrama de casos de uso	interacción usuario-sistema
Diagrama de secuencia	comunicación entre objetos
Diagrama de actividades	flujo de procesos
Diagrama de estados	comportamiento de objetos

El **diagrama de clases** es el más utilizado para diseño orientado a objetos .

Diagrama de clases

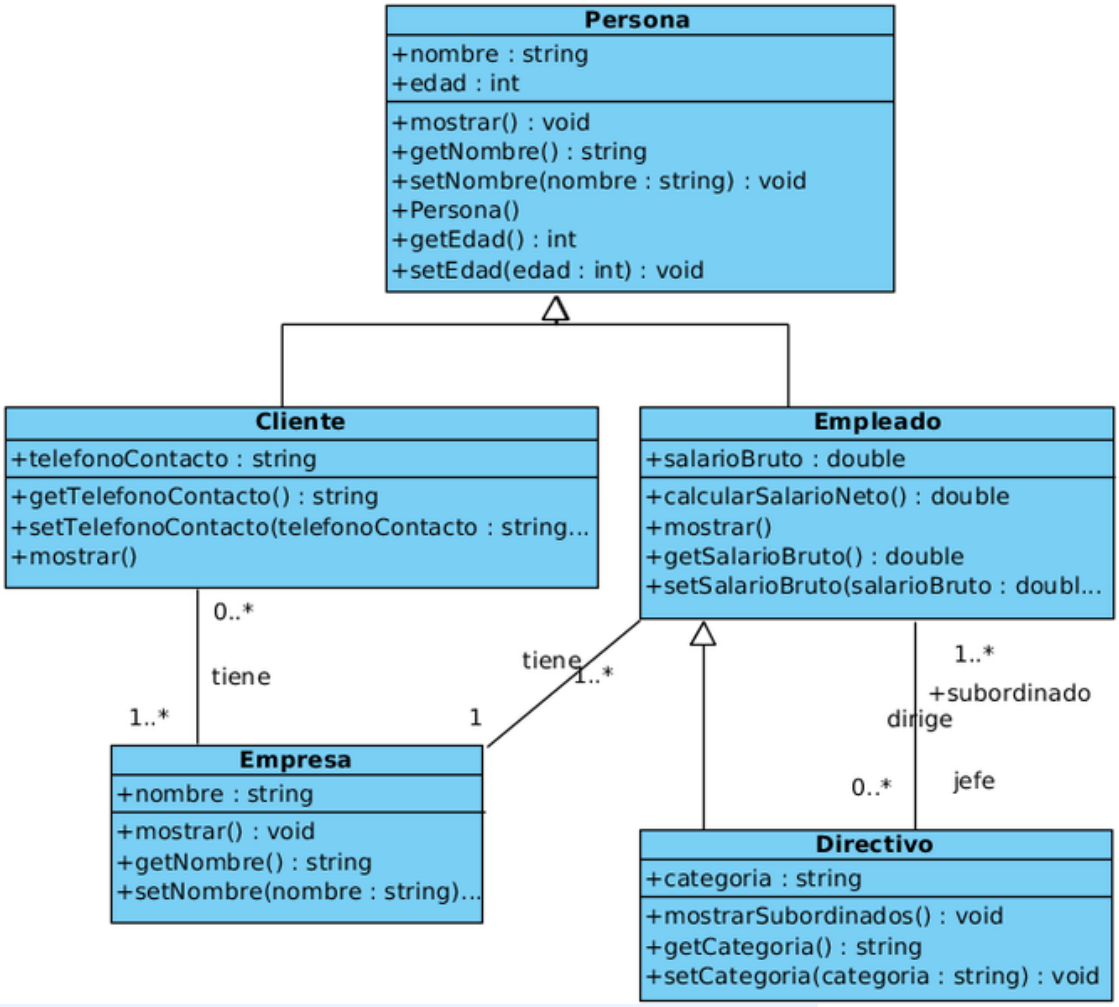
Un **diagrama de clases** es un tipo de diagrama UML que representa:

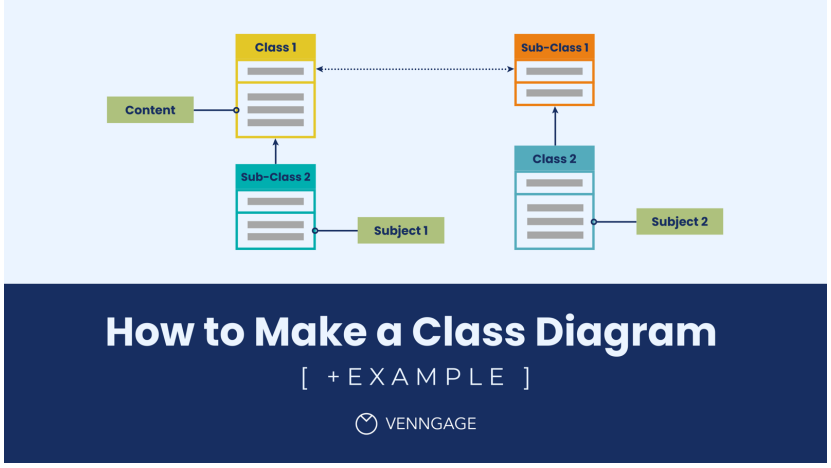
- clases
- atributos
- métodos
- relaciones entre clases

Es básicamente **el plano estructural del software**.

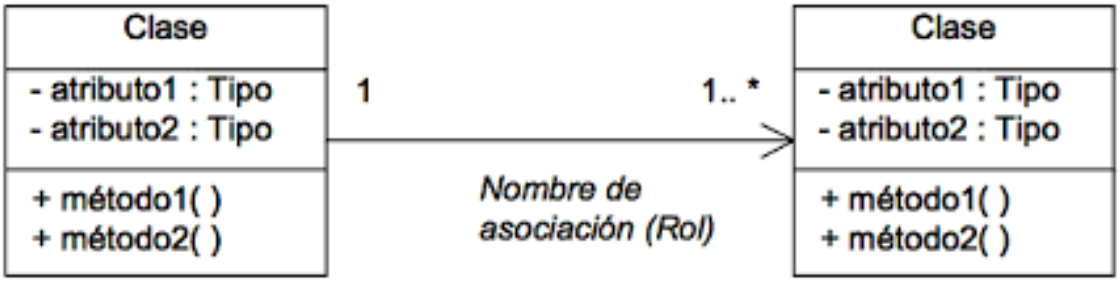
Concepto	Analogía
Clase	molde o plantilla
Objeto	instancia de la clase
Atributo	información
Método	comportamiento
Relación	conexión entre clases

Estructura de una clase en UML





Una clase se representa con **un rectángulo dividido en tres partes**.



Clase

Una **clase** es una plantilla que define:

- propiedades (atributos)
- comportamientos (métodos)

Ejemplo
Clase:

- Libro
    - Representa todos los libros del sistema.

Objeto

Un **objeto** es una instancia concreta de una clase.

Ejemplo

- Clase:
    - Libro
- Objeto:
    - Libro: "El Quijote"

Atributos

Un **atributo** es una variable que almacena información sobre una clase.

Ejemplo:

- Clase: Libro

- Atributos:
  - Titulo
  - Autor
  - ISBN
  - annoPublicacion

Tipos de atributos

Tipo	Ejemplo
Identificador	idUsuario
Texto	nombre
Numérico	precio
Booleano	disponible

Métodos

Un **método** es una función que define las acciones que puede realizar una clase.

Ejemplo:

- Clase Usuario
  - registrar()
  - reservarLibro()
  - cancelarReserva()

Comparación atributo vs método

Concepto	Explicación	Ejemplo
Atributo	información	nombre
Método	acción	registrar()

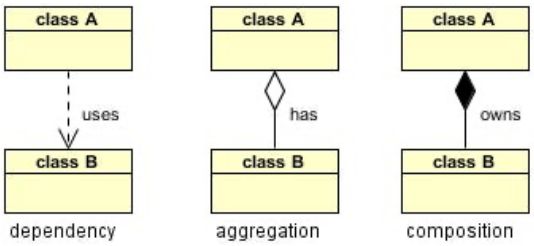
Visibilidad en UML

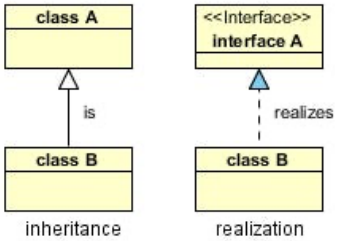
En UML se utilizan símbolos para indicar el acceso a atributos y métodos.

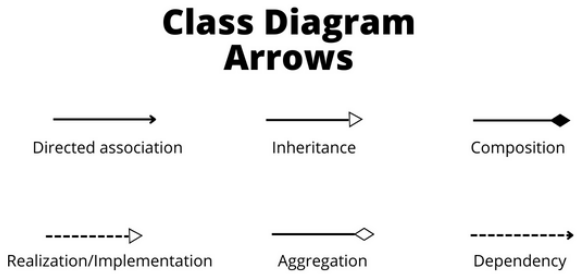
- público +
- privado –
- **protegido #**

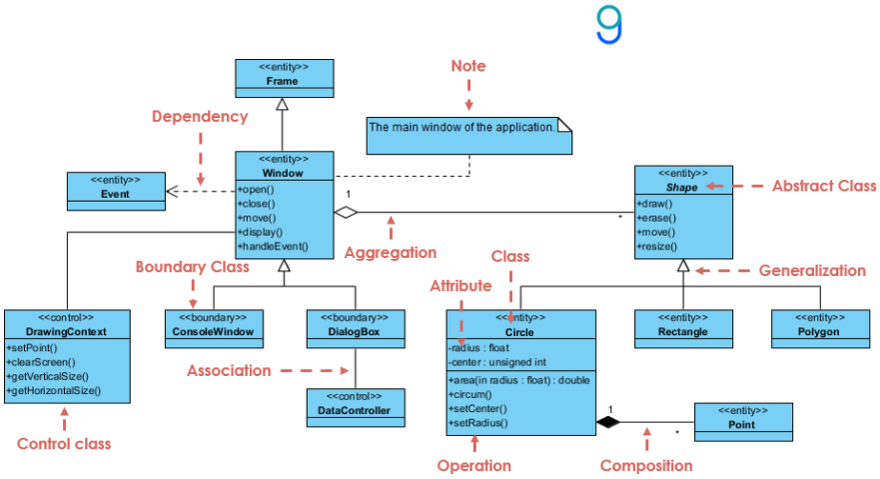
Ejemplo:
+ nombre
- contraseña
edad

Relaciones entre clases









Las relaciones muestran **cómo interactúan las clases**.

Las más importantes son:

- asociación
- herencia
- agregación

- composición

Estas relaciones explican cómo colaboran las clases dentro del sistema .

Asociación

La **asociación** es una relación general entre dos clases que interactúan.

Ejemplo:
Usuario ---- Libro

Significa:
un usuario puede reservar libros

Multiplicidad

La multiplicidad indica **cuántos objetos se relacionan**.

Notación	Significado
1	uno
0..1	opcional
1..*	uno o muchos

Ejemplo:
Usuario 1 ----- * Libro

Herencia

La **herencia** es una relación donde una clase hija hereda características de una clase padre.

Ejemplo:

Vehiculo
↑
Coche

Significa:
Coche es un Vehiculo

Agregación

La **agregación** representa una relación débil donde una clase contiene a otra.

Ejemplo:
Equipo ◇---- Jugador

Si el equipo desaparece:
los jugadores siguen existiendo

Composición

La **composición** es una relación fuerte donde una clase depende totalmente de otra.

Ejemplo:
Pedido ◆---- ProductoPedido

Si el pedido se elimina:
los productos también

Comparación de relaciones

Relación	Tipo	Ejemplo
Asociación	vínculo general	usuario-libro
Herencia	especialización	coche-vehiculo
Agregación	relación débil	equipo-jugador
Composición	relación fuerte	pedido-producto

Caso práctico: Biblioteca digital

Clases principales

- Libro
- Usuario
- Reserva

Clase Libro
Atributos:

- Titulo
- Autor
- ISBN
- annoPublicacion

Clase Usuario

Atributos:

- nombre
- email
- idUsuario

Métodos:

- registrar()
- reservarLibro()
- cancelarReserva()

Relación
Usuario 1 ---- * Libro

Ventajas de usar UML

Ventaja	Explicación
Diseño previo	pensar antes de programar
Comunicación	todos entienden el sistema
Detección de errores	problemas antes del código
Documentación	arquitectura clara

Herramientas para UML

Herramienta	Uso
Draw.io	gratuita
Lucidchart	colaboración
StarUML	generación de código
Visual Paradigm	profesional

Actividad práctica

Diseñar un sistema de **tienda online**.

Clases posibles:
- Cliente
- Producto
- Pedido

Preguntas:
- ¿Qué atributos tendría Cliente?
- ¿Qué métodos tendría Pedido?
- ¿Qué relaciones existirían?

Resumen

- **UML**
  - Lenguaje estándar para modelar software.

- **Diagrama de clases**
  - Representa la estructura del sistema.

  - **Elementos principales**
    - clase
    - atributo
    - método

    - relaciones

- **Relaciones**
  - asociación
  - herencia
  - agregación
  - composición

El diagrama de clases permite **diseñar la arquitectura del sistema antes de programar**.